University of Pisa

Parallel and Distributed Systems

# Optimization of Partitioned Hash Join with Duplicates

Assignment 2 - Report

Gianluca Panzani

April 16, 2026

# Contents

**1 Implementation** **1**
1.1 Hash Function Choice . . . . . 1
1.2 Benchmark . . . . . 1
1.3 Checker . . . . . 2

**2 Parallelization Strategy** **2**

**3 Results Analysis** **2**
3.1 Join and Partition Time . . . . . 3
3.2 Strong Scaling . . . . . 4
3.3 Weak Scaling . . . . . 5
3.4 Speedup . . . . . 5

**4 Conclusions** **6**

# 1 Implementation

The repository is organized around one sequential baseline and four optimized variants of the same partitioned hash-join pipeline. The core source files are:

- `hashjoin_seq.cpp`: sequential reference;
- `hashjoin_par_p.cpp`: parallel partition, sequential join;
- `hashjoin_par_pj.cpp`: parallel partition + parallel join (contiguous partition ranges);
- `hashjoin_par_pj_wb.cpp`: adds workload-balanced assignment of partitions;
- `hashjoin_par_pj_wb_map.cpp`: same balanced scheduling, plus flatter memory layout for partition counters/offsets and a custom flat hash table in the join build phase.

All executables preserve the same algorithmic structure:

1. deterministic generation of relations `R` and `S`;
2. partitioning of both relations (histogram, exclusive prefix sum, scatter);
3. local build/probe join on matching partition ids;
4. reduction into `join_count`, `checksum1`, `checksum2`.

Keeping this structure unchanged across versions is important because it allows direct and fair comparisons between optimization steps. Phase timings are exported separately (partition, join and total), so each code change can be analyzed also at phase level.

## 1.1 Hash Function Choice

In Module 1, the simple `mask` mapping was the fastest option in my tests. However, in this module I preferred not to keep the same mapping used in the provided reference-style code, so I switched to an xorshift-based mapping before masking (`key ^= key >> ...; pid = key & (P-1)`).

I did not choose the `fmix32fold`-style mapping here because it is heavier in arithmetic operations and, in previous experiments, it gave worse performance than xorshift for this workload.

## 1.2 Benchmark

Benchmark automation is implemented in `benchmark.sh`. The script takes an executable target, loads a grid file, compiles the selected target with `make -B` and executes all combinations via Slurm (`run_with_slurm.sh`). For each run, the executable appends one CSV row containing configuration parameters, phase times, throughputs and correctness outputs.

In the current campaign:

- fixed values are `nr=50'000'000`, `ns=50'000'000`, `P=256`, `seed=13`, `max_key=1'000'000`;
- while `partition_threads` and `join_threads` vary over $\{4, 8, 16, 32, 64\}$;
- the sequential baseline runs with one thread.

The Slurm runner (`runners/run_with_slurm.sh`) runs a single combination of parameters of the grid and pins execution to `node05`. This keeps machine-level variability under control and allows direct comparison of different implementations under the same hardware conditions.

### 1.3 Checker

Correctness checks are handled by `checker.cpp`. The checker scans the output directory (`out/` by default), reads all regular files, and compares their raw content byte-by-byte against the first file. The logic is intentionally strict: if at least one file differs, it reports the mismatch and returns a non-OK status.

In this project, each Slurm output file stores the final triple: `join_count`, `checksum1`, `checksum2`. Therefore, identical files imply identical final join outputs across compared runs. In the collected data, all variants produced the same values, which confirms that optimization changes preserve functional correctness.

## 2 Parallelization Strategy

Parallelization is introduced phase-by-phase, without changing the global algorithm.

**Partition phase (parallelized in all parallel variants).** For each relation, the partition pipeline keeps the standard three stages:

1. *Histogram:* input is split into chunks and each thread builds a local histogram over $P$ partitions.
2. *Prefix sum:* local histograms are reduced into a global histogram, then an exclusive prefix sum computes global partition starts. In the current code this step is **not parallelized**: it is executed as a serial scan over the $P$ counters, followed by a serial construction of per-thread offsets.
3. *Scatter:* each thread writes its chunk into the output using thread-specific offsets derived from the prefix stage.

The key design detail is that scatter uses precomputed thread-local offsets, so the hot loop does not require atomics or locks.

**Join phase.** After partitioning, partitions are independent units of work.

- `hashjoin_par_p`: join is still sequential; only partition is parallel.
- `hashjoin_par_pj`: partitions are split into contiguous ranges across join threads.
- `hashjoin_par_pj_wb`: partitions are assigned with greedy load balancing based on their size.
- `hashjoin_par_pj_wb_map`: same balanced scheduling, plus a flat open-addressing count table in build/probe and flat arrays in partition counters/offsets.

**What remains mostly sequential.** The global prefix-sum step and the final reduction of per-thread join results are lightweight compared with histogram/scatter/probe loops, so they are kept simple. In practice, with $P = 256$, this serial part has low absolute cost compared with the full partition and join loops. This is coherent with the measured behavior: once partition and join loops are parallelized, remaining bottlenecks come mostly from memory traffic and per-thread overhead at high thread counts, not from control-heavy serial code.

## 3 Results Analysis

All plots in this section are generated from the current benchmark CSV files in `src/results/`. The sequential baseline mean total time is 4.191 s.

## 3.1 Join and Partition Time

The phase-level matrices show the central behavior of the implementation.

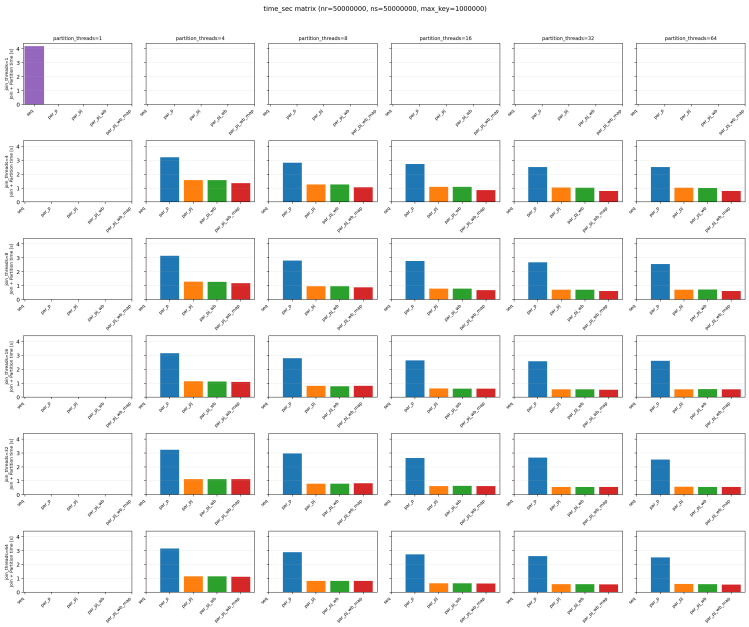


Figure 1: Total execution-time (`time_sec`) matrix for $nr = ns = 50\text{M}$, $P = 256$, $max_key = 10^6$.

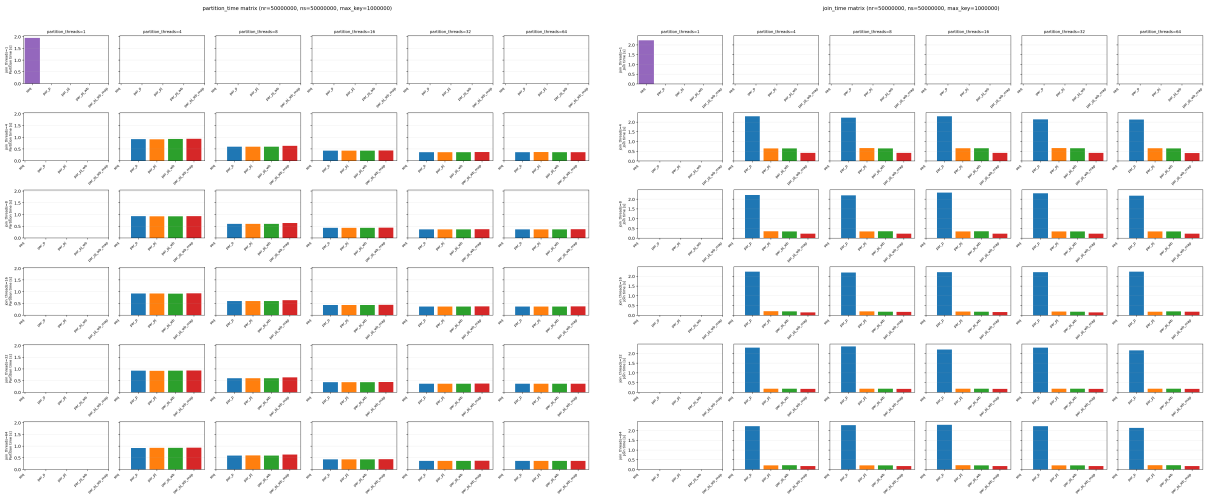


Figure 2: Partition-time and join-time matrices for $nr = ns = 50\text{M}$, $P = 256$, $max_key = 10^6$.

Partition time scales well from 4 to 32 threads, then flattens:

- around 0.92 s at 4 partition threads;
- around 0.36–0.37 s at 32–64 partition threads.

This indicates good early scaling followed by memory-system saturation.

Join time is where variants differ the most. The executables `par_pj` and `par_pj_wb` reduce join time to about 0.18 s in the best area. The `par_pj_wb_map` variant improves further, reaching 0.13 s (best run, 32 partition threads and 16 join threads), confirming that data-structure locality in build/probe is a major factor.

## 3.2 Strong Scaling

Strong scaling is measured by fixing input size and increasing threads.

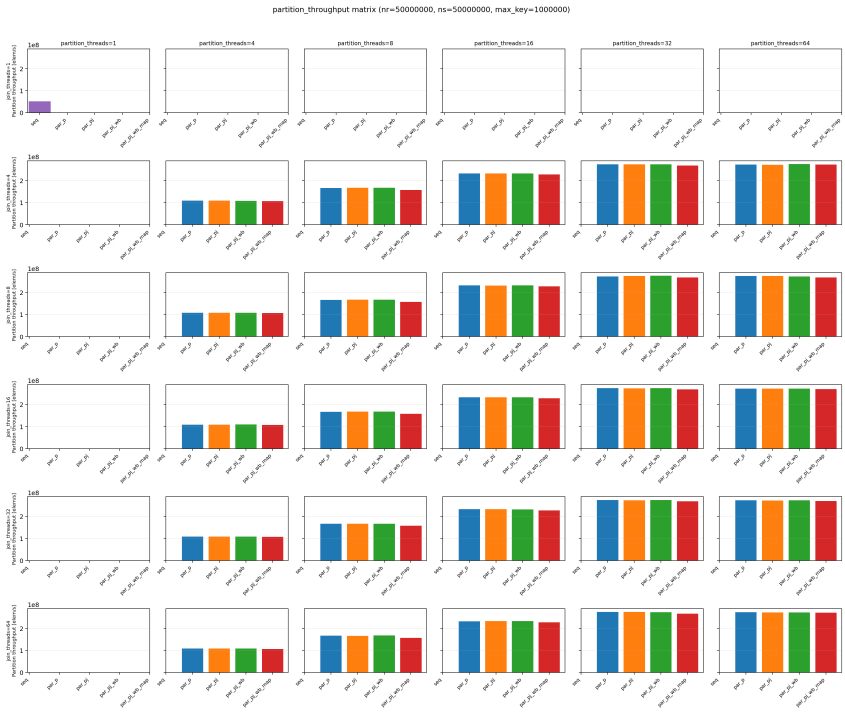


Figure 3: Partition-throughput matrix under fixed problem size.

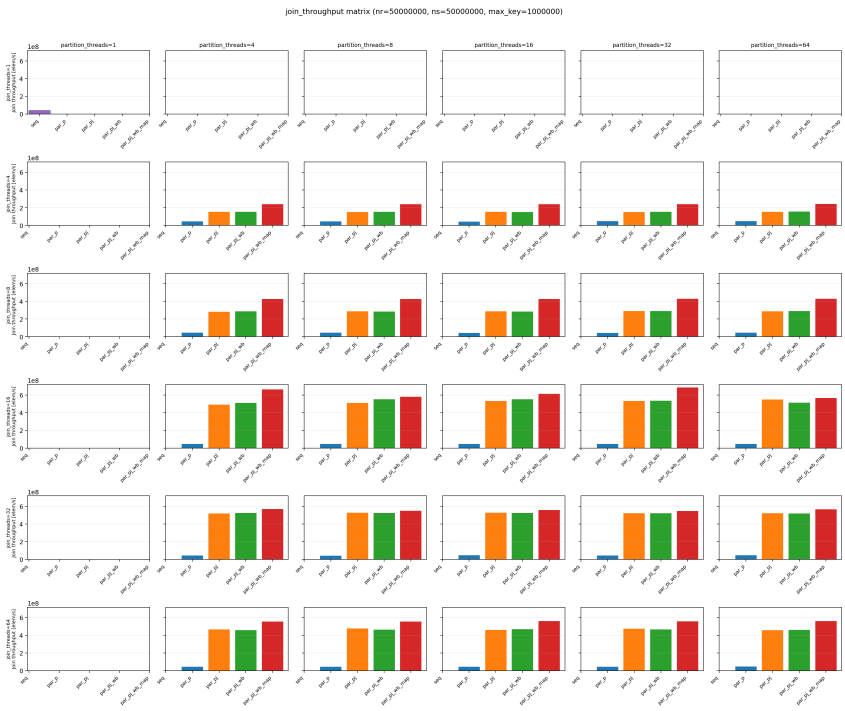


Figure 4: Join-throughput matrix under fixed problem size.

On the diagonal ($T_{part} = T_{join}$), `par_pj_wb_map` reaches:

- speedup 3.09× at 4 threads;

- speedup 6.87× at 16 threads;
- speedup 7.62× at 64 threads.

Scaling is strong up to about 32 threads, then gains are smaller. This is coherent with the partition plateau and with increased overhead/imbalance sensitivity in the join stage at high thread counts. In fact, the best result is got with `par_pj_wb_map` using 16 join threads and 32 partition threads, with a speedup of about 8.26×.

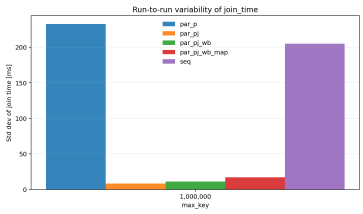


Figure 5: Standard deviation of `join_time`.

## 3.3 Weak Scaling

Strict weak scaling requires increasing input size proportionally to the number of threads while keeping work per thread constant. The current campaign uses a fixed size ($nr = ns = 50$M), so it is not a full weak-scaling experiment.

To keep this section grounded on available data, the global throughput matrix can be read as a scaling *proxy* while resources increase:

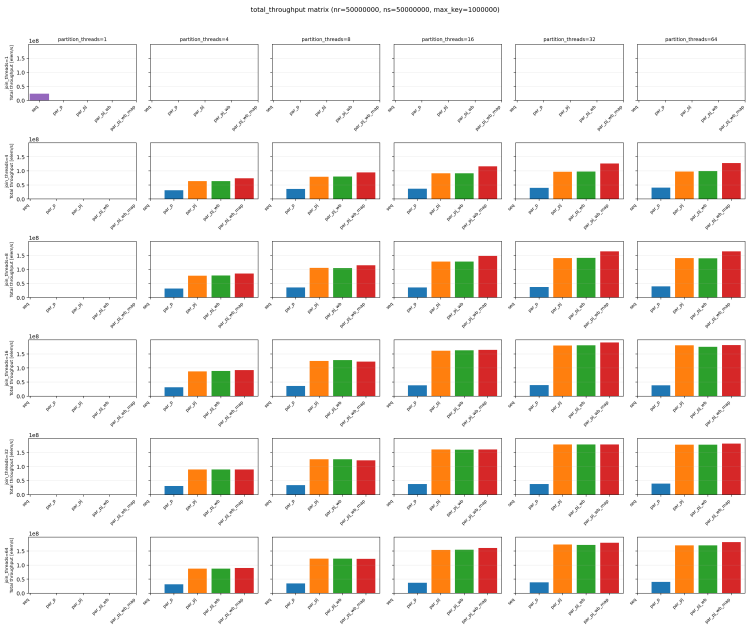


Figure 6: Total throughput matrix used as weak-scaling proxy under fixed $N$.

The figure shows higher throughput as thread counts increase up to the 32–64 range, with reduced marginal gains at the top end. A true weak-scaling study still requires new runs where $N$ grows with thread count ($N \propto T$).

## 3.4 Speedup

Global speedup is computed against the sequential mean total time (4.191 s).

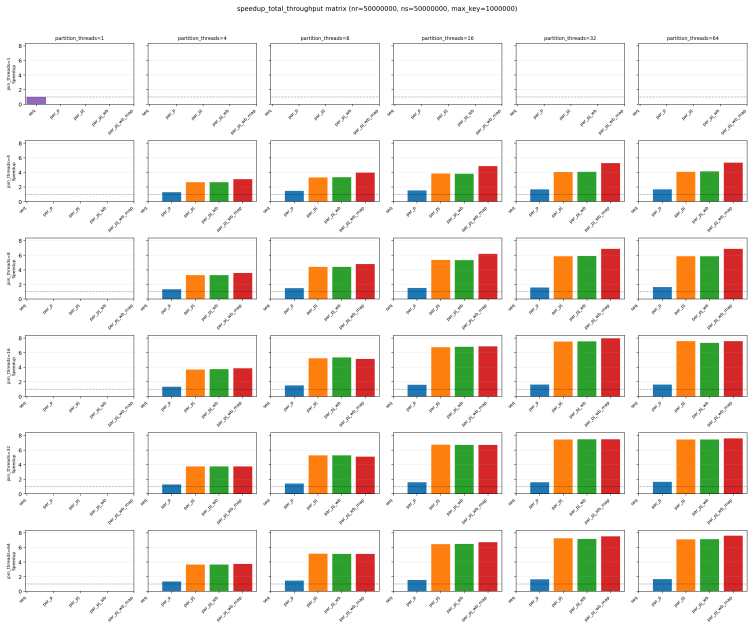


Figure 7: Total speedup matrix over thread combinations and variants.

| Variant | Best $T_{part}$ | Best $T_{join}$ | Best time [s] | Speedup |
|---|---|---|---|---|
| `hashjoin_par_p` | 64 | 32 | 2.394 | 1.75× |
| `hashjoin_par_pj` | 64 | 16 | 0.549 | 7.63× |
| `hashjoin_par_pj_wb` | 32 | 16 | 0.545 | 7.69× |
| `hashjoin_par_pj_wb_map` | 32 | 16 | 0.507 | 8.26× |

Table 1: Best observed configuration per executable (baseline: sequential mean total time).

The table confirms the optimization trajectory: partition-only parallelization gives limited gains, while parallel join plus balanced scheduling gives the main jump; flatter data structures in `par_pj_wb_map` provide the final improvement.

# 4 Conclusions

The results show a clear progression:

- parallelizing only partitioning is not sufficient (speedup capped around 1.75×);
- parallelizing join is the dominant improvement (around 7.6×);
- workload balancing improves robustness of high-thread runs;
- locality-oriented structures (`par_pj_wb_map`) deliver the best result (8.26×).

So, for this workload, the main insights are that join-phase optimization dominates total performance, while partition scaling saturates earlier and sets a practical upper bound unless data movement costs are further reduced. A future extension should add a true weak-scaling campaign ($N \propto T$) to complete the scalability analysis requested by the assignment.